

```

/*
=====
Exploration et analyse du dataset
Objectif :
Explorer les données du dataset Olist afin de mieux
comprendre les comportements clients, les ventes,
les produits et les commandes avant la création des
indicateurs métier et du dashboard Power BI.
--Répartition des statuts de commande
SELECT order_status,
       COUNT(*) AS Total_orders,
       ROUND(
           COUNT(*)* 100.0/
           (SELECT COUNT(*) FROM orders),2
       ) AS Pourcentage_orders
FROM orders
GROUP BY order_status
ORDER BY Total_orders DESC
--Classement des régions les plus actives
SELECT customer_state,
       COUNT(*) AS Total_orders,
       RANK ()OVER(ORDER BY COUNT(*) DESC) AS State_rank
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_state
LIMIT 10
--Classement des régions par chiffre d'affaires--
SELECT c.customer_state,SUM(oi.price)AS CA,
RANK()OVER (
           ORDER BY (SUM(oi.price))DESC
       ) AS State_rank
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
JOIN order_items oi
ON o.order_id = oi.order_id
GROUP BY c.customer_state
LIMIT 10
-- Nombre moyen d'articles par commandes--
SELECT AVG(nb_articles) AS AVG_nb_articles,
       MAX(nb_articles)AS MAX_nb_articles,
       MIN(nb_articles) AS MIN_nb_articles
FROM (
       SELECT order_id, COUNT(*) AS nb_articles
       FROM order_items
       GROUP BY order_id)AS commandes
--Analyse de la saisonnalité des commandes--
WITH cmd_mois AS (
SELECT
DATE_TRUNC('month', order_purchase_timestamp) AS periode,
COUNT(*) AS total_commandes
FROM orders
GROUP BY DATE_TRUNC('month', order_purchase_timestamp)

```

```

)
SELECT
periode,
total_commandes,
LAG(total_commandes) OVER (
ORDER BY periode
) AS periode_precedente
FROM cmd_mois
ORDER BY periode;
-- Respect des delais de livraison--
WITH Delivery_table AS(
    SELECT TO_CHAR(order_estimated_delivery_date, 'YYYY/MM/DD') AS
Delivery_estimated,
            TO_CHAR(order_delivered_customer_date, 'YYYY/MM/DD') AS
Delivered_date
FROM orders
    WHERE order_status = 'delivered'
),
Status_table AS (
    SELECT Delivery_estimated,
            Delivered_date,
CASE retard'
avance'
    WHEN Delivered_date > Delivery_estimated THEN 'En
    WHEN Delivered_date < Delivery_estimated THEN 'En
    ELSE 'A temps'
END AS Delivery_status
FROM Delivery_table
)
SELECT Delivery_status,
COUNT(*) AS nb_commandes,
ROUND(
    COUNT(*)*100/SUM(COUNT(*))OVER(),2) AS prct_commandes
FROM Status_table
GROUP BY Delivery_status
-- Catégories au dessus de la moyenne globale--
WITH Avg_prix_categorie AS (
    SELECT p.product_category_name AS Categorie,
            AVG(oi.price) AS Avg_prix
FROM Products p
JOIN order_items oi
ON p.product_id = oi.product_id
GROUP BY Categorie
)
SELECT Categorie,
Avg_prix
FROM Avg_prix_categorie
WHERE Avg_prix > (
)
SELECT AVG(price)
FROM order_items

-- TOP 10 Clients --
WITH CA_client AS (

```

```

        SELECT c.customer_unique_id AS clients,
               SUM(oi.price) AS Total_CA
        FROM customers c
        JOIN orders o
        ON c.customer_id = o.customer_id
        JOIN order_items oi
        ON o.order_id = oi.order_id
        GROUP BY clients
    )
    SELECT clients,
           Total_CA,
           DENSE_RANK () OVER(ORDER BY Total_CA DESC) AS TOP_10
    FROM CA_client
    LIMIT 10;

*/
-- Distribution de prix par tranche --
WITH Table_tranches AS (
    SELECT price,
           CASE
               WHEN price < 50 THEN 'Faible'
               WHEN price BETWEEN 50 AND 100 THEN 'Moyen'
               WHEN price BETWEEN 100 AND 200 THEN 'Elevé'
               ELSE 'Tres Elevé'
           END AS Tranche_prix
    FROM order_items
)
SELECT Tranche_prix,
       COUNT(*) AS Nmbr_ventes,
       ROUND(COUNT(*)*100.0/SUM(COUNT(*) OVER(),2) AS Pourcentage
FROM Table_tranches
GROUP BY Tranche_prix
ORDER BY Nmbr_ventes DESC

```